

Millenium Falcon az aszteroidamezőben

Bevezetés

Han Solo és Chewbacca a Kessel-futam során rossz helyen lépett ki a hiperútból és egy aszteroidamezőben kötött ki. Az új ugrás csak a mezőn kívül oldható meg, hiszen fénysebességgel felkenődni egy sziklán még fájdalmasabb mint lassabban. Tehát Han Solonak előbb ki kell vinnie a Millenium Falcont az üres űrbe.

Figure 1: Millenium Falcon az aszteroidamezőben

Háttér: könyvtári modulok

A feladathoz szükség lesz az általunk készített három source file és a hozzájuk tartozó header file-okra. Ezek a következők:

1. *terminal_draw*: Konzolos rajzoláshoz biztosít szolgáltatásokat. Tartalmazza a *Color* enumon kívül a következő függvényeket.
 - *void clear_screen()*: Ez a függvény törli a terminál ablak teljes tartalmát.
 - *void get_screen_size(int * width, int * height)*: Ez a függvény beállítja a megadott mutatók által hivatkozott változók értékét a terminál ablak mérete alapján rendre az egy sorba és egy oszlopba kiírható karakterek számára.
 - *void draw_char(int x, int y, char c, Color color)*: A függvény kiírja a megadott karaktert a megadott színben az (x, y) koordinátába. A (0, 0) pont az ablak bal felső sarkát jelzi.
 - *void sleep_ms(int milliseconds)*: Felfüggeszti a program futását a megadott ezredmásodpercre.
2. *keyboard_listener*: Egy aszinkron szálon futó, a billentyűzetet figyelő függvényt tartalmaz, egy kilépést jelző karaktert (*QUIT*) és egy *Action* enumerációt, melyet az űrhajó irányításánál tudsz felhasználni.
 - *void start_keyboard_listener_async(Action * c)*; hívásakor indít egy új szálát, mely a billentyűzetről olvas be karaktereket és tárolja le a paraméterként kapott memóriaterületre. Ezt a memóriaterületet

olvashatja a hívó (*main*) függvény a főszálon és vezérelheted vele az űrhajót. Használata egyszerűen – másolhatod is a *main* függvényedbe és a továbbiakban a pointer változót használhatod:

```
Action* action = malloc(sizeof(Action));  
...  
start_keyboard_listener_async(action);
```

3. *falcon*: A header tartalmazza az űrhajó pozícióját tartalmazó *Ship* struktúrát és a Millenium Falcont kirajzoló, a source file-ban implementált függvényt is.

Programozási feladat

A játékban egy űrhajót, a Millenium Falcont irányítjuk horizontális, azaz x-irányban, a *keyboard_listener* header-ben megadott *Action* enumban megadott karakterek segítségével. Ezzel a módszerrel a terminál legelső sorából vertikálisan lefele mozgó aszteroidákat tudjuk kikerülni. A játék véget ér, ha az űrhajó ütközik egy aszteroidával.

Az alábbi részfeladatokat kell megvalósítanod:

1. *main*: A főprogram kezeli a játék futását. A feladatai: elindítja a keyboard figyelő függvényt (lásd a fenti példakódot), kéri el a terminál adatait és egy végtelen ciklusban takarítja a terminált és rajzolja ki a Millenium Falcont és az aszteroidákat, valamint az ütközést ellenőrző függvény válasza alapján szakítja meg a futást, továbbá emberi szintre lassítva a *sleep_ms* függvénnyel (20 – 100 ms intervallum az emberi). Az űrhajó kezdeti helyzete vertikálisan a terminál alja, horizontálisan a közepe. **(10 pont)**
2. A *falcon* header és source file továbbfejlesztése az űrhajó mozgatásáért felelős *move_ship* függvénnyel. Bemeneti paraméterként a hajó pozícióját és a *user* által adott *Action** típusú paramétert kell átadni. Természetesen a hajó nem mehet ki a terminálról. **(8 pont)**
3. *Asteroid* struktúra: Tartalmazza egy aszteroida horizontális és vertikális pozícióját, valamint a vertikális sebességét és az aszteroidát leíró karaktert is. **(4 pont)**
4. *Asteroids* struktúra: Az *Asteroid* struktúrákból álló tömb burkoló típusa, mely az összes aszteroidát tartalmazza, valamint az aszteroidák mennyiségét és a kapacitást. A kapacitás a már lefoglalt memóriaterületet jelenti, míg a mennyiség az ebből elhasználtat, ami a terminálon jelen lévő aszteroidákkal (nem tökéletesen, de) korrelál. **(4 pont)**
5. *init_asteroids* és *destroy_asteroids*: Első beállítja az aszteroidák kezdő értékeit – az aszteroidák kezdő mennyisége zéró, a kapacitás egy általad választott értékre állítsd be (10 – 100 aszteroida lehet a terminálon a futás későbbi szakaszában). Ezzel a kapacitással dinamikusan foglaljon memóriát. A *destroy_asteroids* felszabadítja a memóriát és nullázza az értékeket. **(4 és 4 pont)**
6. *move_draw_asteroids*: Mozgatja az aszteroidákat vertikálisan és egyből ki is rajzolja (rajzoláshoz a *terminal_draw* könyvtár *draw_at* függvényét használva). Ha az aszteroida kiér a terminálról, nem kell kirajzolni. Érdemes a terminálról kilépő aszteroidákat megjelölnöd, például a sebesség 0 értékre vagy a vertikális helyzet –1 értékre állításával. **(6 pont)**
7. *update_asteroids*: Ez a függvény készíti az új aszteroidákat, dinamikusan. Mivel nagyon sok aszteroida lenne, ha minden iterációban generálnánk egyet, érdemes véletlenszerűen megoldani. Azaz, általad szabadon választott 30 – 50 % valószínűséggel készíteni egy új aszteroidát.
Az *Asteroids* objektumod kezelésére a következő hatékony módszert kell megalkotnod, két fontos szempontot figyelembe véve. Először is, a terminálról kisodródott aszteroida "törölhető". Másodsorban, minden új aszteroida készítéséhez újra allokálni a memóriát költséges. Ezért a függvényed egy új aszteroida készítésénél
 - vizsgálja, hogy van-e a terminálról kilépett aszteroida: ha igen, annak a helyére tárolja az új aszteroidát
 - ha nincs kisodródott aszteroida, megvizsgálja, hogy a van-e szabad férőhely az *Asteroids* objektumodban, azaz az aszteroidák mennyisége a kapacitásnál kevesebb és ha igen, úgy a mennyiséget növelve oda tárolja az új aszteroidát
 - ha elfogyott a kapacitás, újra kell foglalni a memóriát egy (makróban) rögzített értékkel és az előző pontban leírtak szerint eljárni.

Figyelj a dinamikus memóriakezelésre! **(12 pont)**

8. *check_collision*: Ellenőrzi, hogy valamelyik aszteroida és az űrhajó koordinátái egybeesnek. Paraméterként az űrhajó és az aszteroidákat tartalmazó objektumokat szükséges átadni. A hajó mérete, mint a *draw_mfalcon* függvényben láthatod, egy 5×5 négyzet. **(8 pont)**

Elvárások a programmal szemben

1. A megoldáshoz *semmilyen* segédeszköz nem használható, kivéve a C referenciát.
<https://en.cppreference.com/w/c.html>
2. A program végleges verziójának működőképesnek kell lennie. Forduljon és fusson! A nem forduló kód 0 pontot ér!
3. Ne használj globális változókat! Csak a makrók megengedettek!
4. Törekedj a szép, áttekinthető kódolásra, használj indentálást, kerülj a kódismétlést!
5. Ne foglalj szükségtelenül memóriát, és kerülj a memóriaszivárgást!
6. Kerülj a nem definiált viselkedést okozó utasításokat!
7. Ahol nincs pontosan leírva egy feladatrész, annak egyedi megvalósítását rád bízuk.
8. A végleges programban a be nem tartott elvárások pontlevonással járnak!

Megjegyzés

Amennyiben Linuxon, a játék futása után a terminálon nem látod a begépett karaktereket, úgy a "stty sane" utasítással tudod helyreállítani azt.